

COLLEGE OF ENGINEERING, GUINDY

Chennai - 600025



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY - IST

Assignment Report on

Containerization using Docker and Docker Hub

in

**DISTRIBUTED SYSTEMS AND COMPUTING –
IT23601**

Submitted by

Mohan Vishnu Kumar – 2023115026

Containerization of RPC Application Using Docker and Docker Hub

1. Introduction

Docker is a containerization platform that allows applications and their dependencies to be packaged into lightweight, portable containers. These containers run consistently across different environments such as local systems, servers, and cloud platforms.

In this experiment, the RPC-based distributed application developed earlier was containerized using Docker. The Docker image was then uploaded to Docker Hub and successfully pulled and executed again to demonstrate portability and environment independence.

2. Objective of the Experiment

The objectives of this experiment are:

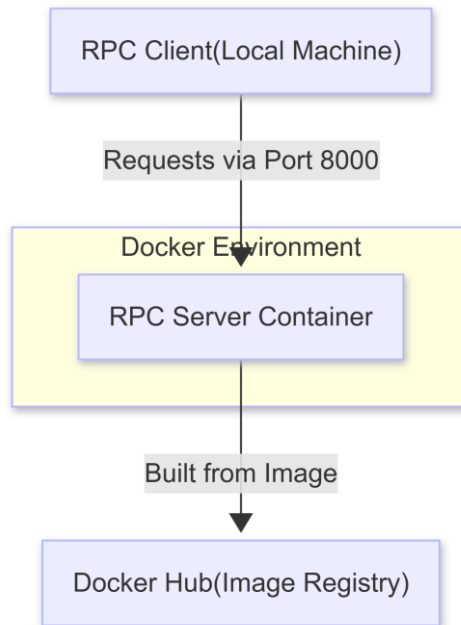
1. To understand Docker containerization.
2. To create a Docker image for an RPC server.
3. To run the application inside a Docker container.
4. To push the image to Docker Hub.
5. To pull and run the image again from Docker Hub.
6. To demonstrate application portability.

3. Tools and Technologies Used

Component	Description
Application	Python RPC Server
Container Platform	Docker Desktop
Registry	Docker Hub
Language	Python
Operating System	Windows 11
Cloud Technology	Containerization

4. System Architecture

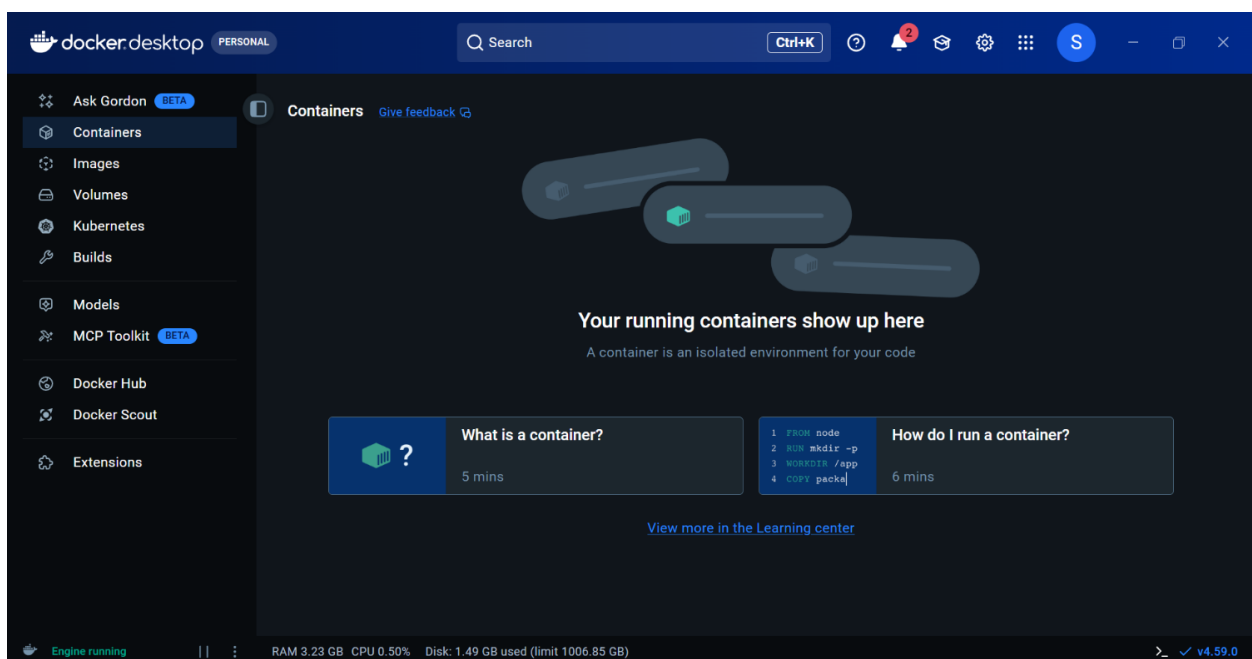
The Docker container hosts the RPC server, and the client communicates with it through exposed ports.



6. Implementation Details

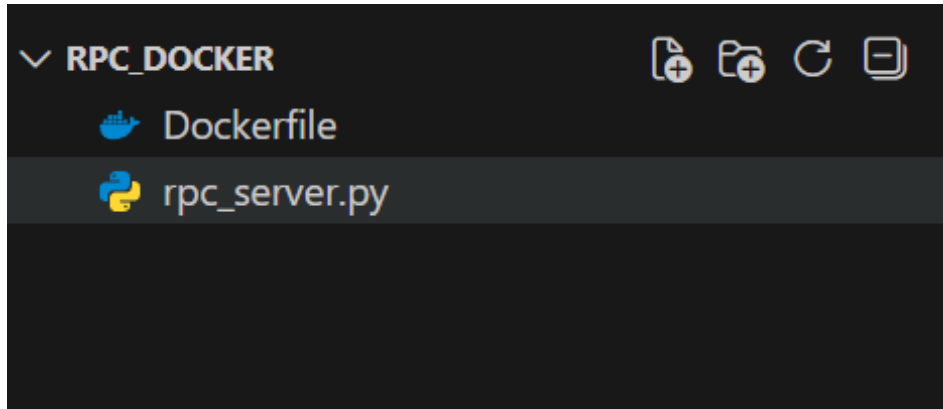
Step 1: Installing Docker

Docker Desktop was installed on Windows 11



Step 2: Project Structure

A project folder was created containing:



Step 3: Creating Dockerfile

The Dockerfile was written to define the environment and execution steps.

```
FROM python:3.10
WORKDIR /app
COPY rpc_server.py .
EXPOSE 8000
CMD ["python", "rpc_server.py"]
```

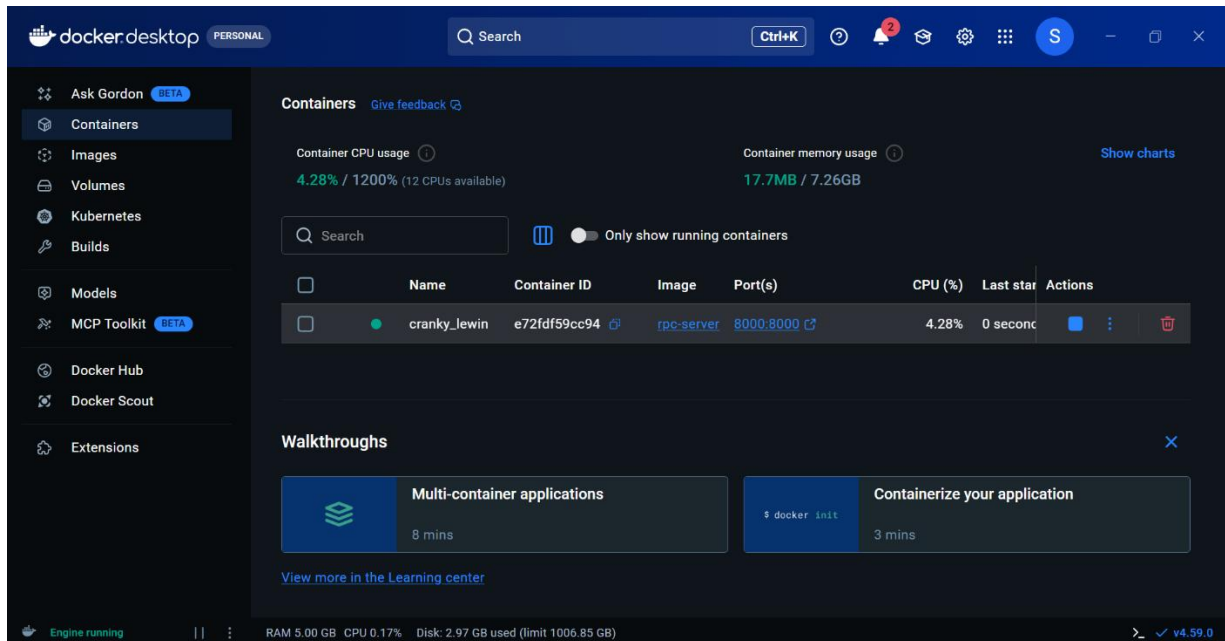
Step 4: Building Docker Image

```
PS C:\Users\curvy\Desktop\RPC_Docker> docker build -t rpc-server .
[+] Building 600.9s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 145B
=> [internal] load metadata for docker.io/library/python:3.10
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/python:3.10@sha256:e0288172e9fe05611095f4b936f326d8de5b542f1f328c66775f5ab71bcd0884
=> resolve docker.io/library/python:3.10@sha256:e0288172e9fe05611095f4b936f326d8de5b542f1f328c66775f5ab71bcd0884
=> sha256:59b9e0b8eb6a6d599d8f3c5cc6789a811bfb993ea3282549194b6b5c2cf28709 21.13MB / 21.13MB
=> sha256:2e6bdfa7547c3302acce3fb889d50acf2d133fb4d782c12304dfef258ecce55c 6.10MB / 6.10MB
=> sha256:128c712640095cd6361adb8c415d18f40180beef843ae18943d3a366993d7749 236.00MB / 236.00MB
=> sha256:b5e2021c4c8bd1a46b34d9608a9381afdc333600ee1ef3c94306ecf7373e1956 67.79MB / 67.79MB
=> sha256:954d6059ca7b9bb9ceb566ca2239e01ef312165659d656753d7dbace7771a591 25.61MB / 25.61MB
=> sha256:ef235bf1a09a237b896b69935c8c8d917c9c6a78b538724911414afc0a96763c 49.29MB / 49.29MB
=> extracting sha256:ef235bf1a09a237b896b69935c8c8d917c9c6a78b538724911414afc0a96763c
=> extracting sha256:954d6059ca7b9bb9ceb566ca2239e01ef312165659d656753d7dbace7771a591
=> extracting sha256:b5e2021c4c8bd1a46b34d9608a9381afdc333600ee1ef3c94306ecf7373e1956
=> extracting sha256:128c712640095cd6361adb8c415d18f40180beef843ae18943d3a366993d7749
=> extracting sha256:2e6bdfa7547c3302acce3fb889d50acf2d133fb4d782c12304dfef258ecce55c
=> extracting sha256:59b9e0b8eb6a6d599d8f3c5cc6789a811bfb993ea3282549194b6b5c2cf28709
=> extracting sha256:e0f7ce34d4ab4fdab71188e35a6c01f512b7399e0588c2f2db40d22828fa219d
=> [internal] load build context
=> => transferring context: 35B
=> [2/3] WORKDIR /app
=> [3/3] COPY rpc_server.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:10c9cbef9c9e0bc146b6bc2d7ba8e2ddb16e4ffe5ec9a22af50dc95753953938
=> => exporting config sha256:c36ae6fff64c6863a1b8781b81c94064f04732dcb615a86fc812e04883530c61
=> => exporting attestation manifest sha256:6d0c0ba433f7b6e65fcbc317bc3360eae00e6c6c6988beeb3a367eb02f94675
=> => exporting manifest list sha256:317d75d4fb2ff094f4d29da9ee8d983486bc7aa112f3ad337c8dd3c1f57b16ab
=> => naming to docker.io/library/rpc-server:latest
=> => unpacking to docker.io/library/rpc-server:latest
```

Step 5: Running Container

```
PS C:\Users\curvy\Desktop\RPC_Docker> docker run -p 8000:8000 surrealsun/rpc-server:v1
172.17.0.1 - - [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 - - [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 - - [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 - - [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 - - [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
```

The RPC server started inside the container.



Step 6: Testing Client

The client was executed and successfully communicated with the Dockerized server.

```
PS C:\Users\curvy\Desktop\RPC_Docker> python rpc_client.py
Addition: 8
Subtraction: 6
Multiplication: 42
Division: 4.0
Division by zero: Error: Division by zero
```

Step 7: Docker Hub Setup

A Docker Hub account was created and login was done using:

```
PS C:\Users\curvy\Desktop\RPC_Docker> docker login
Authenticating with existing credentials... [Username: surrealsun]

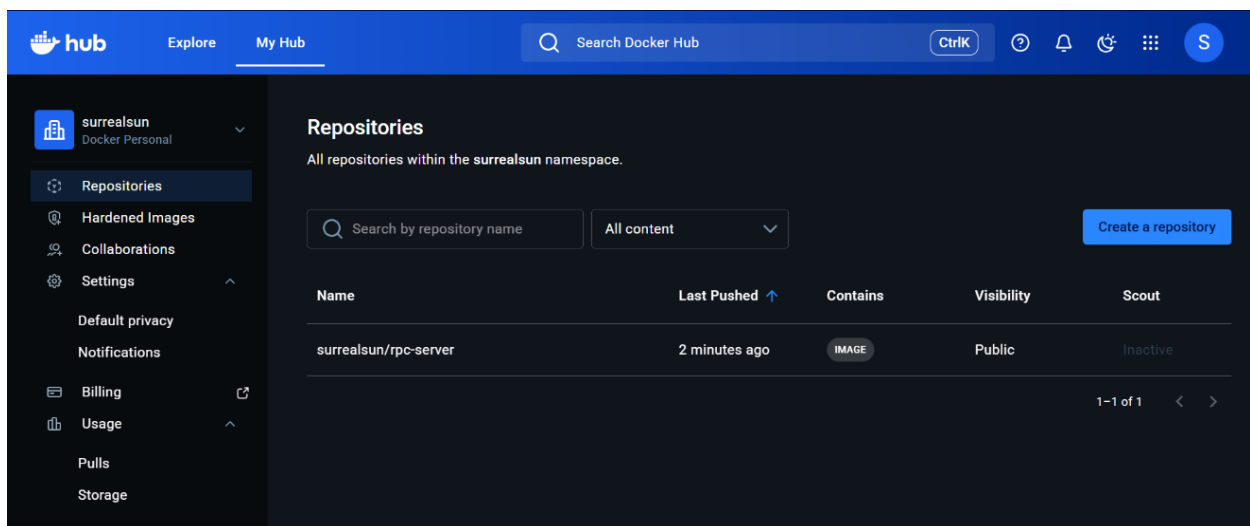
Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

Step 8: Tagging Image & Pushing Image to Docker Hub

```
PS C:\Users\curvy\Desktop\RPC_Docker> docker tag rpc-server surrealsun/rpc-server:v1
PS C:\Users\curvy\Desktop\RPC_Docker> docker push surrealsun/rpc-server:v1
The push refers to repository [docker.io/surrealsun/rpc-server]
56f1cd199a1b: Pushed
b5e2021c4c8b: Pushed
954d6059ca7b: Pushed
128c71264009: Pushed
2585fb086da7: Pushed
ef235bf1a09a: Pushed
59b9e0b8eb6a: Pushed
bc07e6948797: Pushed
e0f7ce34d4ab: Pushed
2e6bdfa7547c: Pushed
v1: digest: sha256:317d75d4fb2ff094f4d29da9ee8d983486bc7aa112f3ad337c8dd3c1f57b16ab size: 856
```

Pushed Image in Docker Hub:



Step 9: Pulling Image from Docker Hub

```
PS C:\Users\curvy\Desktop\RPC_Docker> docker pull surrealsun/rpc-server:v1
v1: Pulling from surrealsun/rpc-server
Digest: sha256:317d75d4fb2ff094f4d29da9ee8d983486bc7aa112f3ad337c8dd3c1f57b16ab
Status: Image is up to date for surrealsun/rpc-server:v1
docker.io/surrealsun/rpc-server:v1
PS C:\Users\curvy\Desktop\RPC_Docker> docker run -p 8000:8000 surrealsun/rpc-server:v1
```

Step 10: Running Pulled Image

```
PS C:\Users\curvy\Desktop\RPC_Docker> docker run -p 8000:8000 surrealsun/rpc-server:v1
172.17.0.1 -- [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 -- [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 -- [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 -- [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
172.17.0.1 -- [05/Feb/2026 19:55:31] "POST / HTTP/1.1" 200 -
```

9. Output

The RPC client successfully invoked remote procedures from the Docker container.

```
PS C:\Users\curvy\Desktop\RPC_Docker> python rpc_client.py
Addition: 8
Subtraction: 6
Multiplication: 42
Division: 4.0
Division by zero: Error: Division by zero
```

9. Conclusion

In this experiment, the RPC server was successfully containerized using Docker. The Docker image was built, executed locally, pushed to Docker Hub, and pulled back for execution. This demonstrates how containerization ensures portability, scalability, and consistency across environments.
